

第一章 绪论

操作系统在计算机系统中占据了特别重要的地位。它是配置在计算机硬件上的第一层软件。而其他软件，如汇编程序、编译程序、数据库管理系统等系统软件，以及大量的应用软件，都将依赖于操作系统的支持。操作系统已经成为现代计算机系统（大、中、小及微型机）、多处理机系统、计算机网络、多媒体系统以及嵌入式系统中都必须配置的、最重要的系统软件。

1.1. 设置操作系统的目的

具有一定规模的现代计算机系统都配备有一个或几个操作系统(OS, Operating System),而且操作系统的性能在很大程度上决定了系统工作的优劣。那么在计算机系统中,设置操作系统的目的是什么呢?

1.1.1 计算机系统的组成

计算机系统包括硬件和软件两个部分。其中,硬件部分也称为裸机,主要包括:CPU、存储系统和各种输入输出设备等构成计算机系统所必需的那部分硬件,或配置在计算机外围的各种特定用途的部件与设备。

但是,仅仅具备了硬件部分而没有软件系统的计算机只不过是一堆废铁,没有任何应用价值。只有配备软件之后,它才能够帮助人类存储和处理信息,例如:显示多媒体文件;浏览 Internet 上的资源;或者从事许多其它有益的工作。

计算机软件大体可以分成两类:管理计算机自身的系统软件 and 为用户执行具体任务的应用软件。系统软件是指计算机系统必须配备的那部分软件。它一般用于管理计算机本身的操作,是对各种领域都通用的软件,例如:各种操作系统、标准程序库、系统维护软件、各种程序设计语言的处理程序等。应用软件则是针对某些特定应用领域配置的软件,例如:科学计算软件、文字处理软件、计算机辅助设计(CAD)软件、计算机辅助教学(CAI)软件、企业管理软件等。

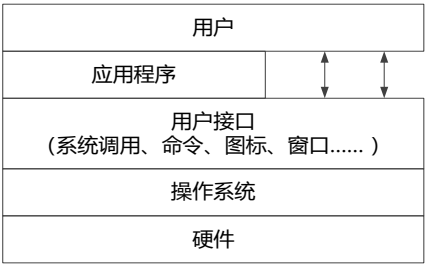


图 1.1: 计算机系统的层级结构

图 1.1 中示出了一个计算机系统的基本组成结构。

在所有计算机系统软件中,操作系统是最基本的系统软件。不论是其他系统软件,还是所有的应用软件都必须在操作系统的管理和支持下运行。操作系统的根本任务就是把系统内包含的所有软、硬件资源有机地组织和管理起来,尽可能充分地发挥它们各自的作用,使整个系统向用户提供他们所需要的各种高质量的服务。同时,它提供一个基础的编程环境,程序员可以在此之上编写应用程序。

1.1.2 操作系统是什么

现代计算机系统是一个非常复杂的系统。即使不考虑效率,只是写一个程序协调所有元件,使它们能够正常工作也是非常困难的。例如,如果每个程序员在编程时都需要考虑磁盘驱动器的工作原理,不得不处理在读取磁盘扇区时可能会发生的几十种错误情况,那么可以肯定,现在的许多程序是编不出来的。

许多年前,人们就清楚地认识到必须寻找一种方法为程序员隐藏硬件细节。这种逐渐演化而成的方法就是:在裸机上覆盖一层软件,以管理系统中的所有元件,为用户(程序员)提供一个易于理解和编程的接口。这层软件就是本课程的研究对象——操作系统。由此,我们给出如下操作系统的定义:

操作系统是计算机系统中的一个系统软件,是一些程序模块的集合——它们能以尽量有效、合理的方式组织和管理计算机的软硬件资源,合理地控制计算机的工作流程,并向用户

提供各种服务功能，使得用户能够灵活、方便、有效地使用计算机，使整个计算机系统能高效地运行，从而在计算机与用户之间起到接口的作用。

下面我们将通过对操作系统的作用和操作系统的的目标这两个方面的分析，来进一步理解什么是操作系统。

1.1.3 操作系统的作用

可以从不同的角度来观察操作系统的作用。从一般用户的角度，可以把操作系统看作是用户与计算机硬件之间的接口；从资源管理的角度，则可把操作系统视为计算机系统资源的管理者，它隐藏了对硬件操作的细节，使用户能更方便地使用机器。

1. 对计算机裸机的扩充

前面提到过，在机器语言级别大多数计算机是难以编程的（特别是编写输入输出程序）。而一般的程序员没有必要知道计算机系统的全部细节。他们对如何针对软盘、硬盘或其它外设编写驱动程序并不感兴趣，只需要一个简单的高层次抽象。比如：程序员只需要知道，磁盘中存放着许多文件，在程序中只需要给出文件名就可以打开文件，之后对其中的信息进行读写，完毕后关闭文件。而在磁盘操作过程中所涉及到的所有细节，在提供给用户的高层抽象中没有出现，程序员甚至感觉不到信息在磁盘中是分块存放的。

那个隐藏了硬盘操作细节，为用户提供面向文件接口的程序就是操作系统。除磁盘外，操作系统还隐藏了系统中许多烦人的细节，涉及中断、计时器、内存管理及其他底层特性。操作系统提供的所有抽象概念都比底层硬件好用得多。也就是说，裸机仅构成计算机系统的物质基础，而实际呈现在用户面前的计算机系统是经过若干层的软件改造过的计算机。裸机在最里层，它的外面是操作系统，经过操作系统提供的资源管理功能和方便用户的各种服务功能，将裸机改造成功能更强、使用更方便的机器。

我们通常把覆盖了软件的机器称为扩充机器，又称之为虚拟机。它向用户提供了一个对硬件资源的抽象模型，用户可利用抽象模型提供的接口使用计算机，而无需了解物理接口实现的细节，从而使用户更容易地使用计算机硬件资源。

2. 作为计算机资源的管理者

现代计算机的硬件由处理器，主存，定时器，磁盘，网络接口，激光打印机和一大堆其它的设备组成。此外还有磁盘中存储着的无数的文件。操作系统的功能是：管理它们，在许多程序竞争使用这些资源时，对资源进行有序分配。

例如，如果在一台机器中有三个程序同时要打印运算结果，那么三个程序的输出将混在一起，最终使打印结果无法使用。而操作系统可以通过在磁盘中缓冲所有程序的输出来避免混乱。打印机在一个任务打印完毕后，才会接受第二个任务。在打印过程中，如果出现了新的任务，操作系统在磁盘的固定区域缓冲，直至打印机空闲（从磁盘中取出新任务送打印机打印）。如此一来，程序可以在任何时刻提交打印任务而绝不会受到丝毫耽搁。虽然打印结果可能不会立即送到打印机，但一切均在操作系统的掌握之中，不会出任何意外。

由于多个用户之间可能会相互干扰，在多用户的计算机系统中，保护内存、I/O 设备和其它资源的任务更加艰巨了。后续章节中将会逐渐展开。

简言之，这种观点认为操作系统的主要工作在于：跟踪和记录谁在使用何种资源，满足合法的资源请求，对不同用户和程序提出的彼此冲突的资源请求进行协调。

1.1.4 操作系统的目标

目前存在着多种不同类型的操作系统，其目标各有所侧重。但是总体来说，在计算机硬件上配置的操作系统，其目标主要有以下几点。

1. 有效性

通常以提高系统资源利用率和提高系统吞吐量两个方面来衡量。在未配置操作系统的计算机系统中，经常会发生这样的情况：

(1) CPU 和 I/O 设备等各种资源，经常处于空闲状态，得不到充分利用；

(2) 内存及外存的空间没有得到合理使用，存放的数据太少或者无序，浪费了大量的存储空间。

而在配置了操作系统之后，可使 CPU 和 I/O 设备由于都保持忙碌状态而得到有效地利用，而且可使得内存和外存中存放的数据因有序而节省大量的存储空间。此外，操作系统还可以通过合理地组织计算机的工作流程，而进一步改善资源的利用率，加速程序的运行，缩短程序的运行周期，从而提高系统的吞吐量。

2. 方便性

一个未配置操作系统的计算机系统是极难使用的。用户要直接在计算机硬件上运行自己的程序，就必须用 0 和 1 这样的机器代码编写程序。如果在计算机硬件上配置了操作系统，用户便可以通过其提供的各种命令来使用计算机系统，极大地方便了用户。

3. 开放性

随着计算机网络的迅速发展，使得计算机操作系统的应用环境已由单机封闭环境转向开放的网络环境。为使来自不同厂家的计算机和设备能通过网络加以集成，并能正确、有效地协同工作，实现应用的可移植性和互操作性，要求操作系统必须提供统一的开放环境。

4. 可扩充性

随着计算机硬件和体系结构的迅速发展，相应地，它们也对操作系统提出了更高的功能和性能方面的要求。操作系统必须具有良好的可扩充性，方能适应计算机硬件、体系结构以及应用快速发展的要求。这就是说，现代操作系统应采用新的结构，以便能够支持方便地增加新的功能和模块，并能修改老的功能和模块。

1.2. 操作系统的形成与发展

操作系统的形成迄今为止已有半个多世纪的时间。在上世纪 50 年代中期出现了单道批处理操作系统；60 年代中期产生了多道程序批处理系统；不久又出现了基于多道程序的分时系统，与此同时也诞生了用于工业控制和武器控制的实时操作系统。20 世纪 80 年代开始至 21 世纪初，是微型机、多处理机和计算机网络高速发展的年代，同时也是微机操作系统、多处理机操作系统和网络操作系统及分布式操作系统的形成和大发展的时代。

1.2.1 无操作系统的计算机系统

1. 人工操作方式

从第一台计算机诞生（1945 年）到 20 世纪 50 年代中期的计算机，属于第一代计算机。此时的计算机利用成千上万的真空管做成，其运行速度仅为每秒数千次，但体积却非常庞大，且功耗也非常高。这时还未出现操作系统，计算机系统是由人工方式直接操纵的。程序员先

将程序写在纸上，然后穿孔制成带有程序信息的卡片。上机时将卡片交给机房的操作员。程序执行结束后结果打印输出，操作员将结果送到输出室，由用户将结果取走。

这种人工操作方式有以下两方面的缺点：

(1) 用户独占全机。在上机过程中，计算机及其全部资源只能由上机用户独占，即：由操作员将穿孔纸带（对应程序和数据）装入纸带输入机，再启动它们将程序和数据读入计算机，然后启动计算机运行。当程序运行完毕并取走结果之后，才能让下一个用户上机。

(2) CPU 等待人工操作。当用户进行装带、卸带等人工操作时，CPU 及内存等资源是空闲的。

这种人工操作方式严重降低了计算机资源的利用率，导致所谓的人机矛盾。而且，随着 CPU 速度的日益提高和系统规模的扩大，这种矛盾变得越发严重。

2. 脱机输入/输出技术

脱机输入/输出技术是指：将事先装有用户程序和数据的数据带装入纸带输入机，在一台外围机的控制下，把纸带上的数据输入到磁带上。当 CPU 需要这些程序和数据时，再从磁带上将其高速地调入内存。类似地，当 CPU 需要输出时，可由 CPU 直接高速地把数据从内存送到磁带上，然后在另一台外围机的控制下，将磁带上的结果通过相应的输出设备输出。

由于程序和数据的输入和输出都是在外围机的控制下完成的，或者说，它们是在脱离主机的情况下进行的，故称之为脱机输入/输出方式。这种方式的主要优点有：

(1) 减少了 CPU 的空闲时间，缓和了人机矛盾。

(2) 提高了 I/O 速度。当 CPU 在运行中需要数据时，是直接从高速的磁带或磁盘上将数据调入内存的，不再是从低速的 I/O 设备上输入，极大地提高了 I/O 速度，从而缓和了 CPU 和 I/O 设备速度不匹配的矛盾，进一步减少了 CPU 的空闲时间。

1.2.2 单道批处理系统

上世纪 50 年代中期发明了晶体管，人们开始使用晶体管替代真空管来制造计算机，从而出现了第二代计算机，彻底改变了计算机的面貌。在硬件上，可靠的晶体管代替真空管组成处理机；磁芯存储器代替继电器存储器。这些新元件的采用大大增强了计算机的可靠性，使计算机从实验室走了出来，人们可以购买一台计算机让它工作，而不用担心它会总出故障。在软件上为了避免程序开发时直接面对机器语言，开发出汇编语言和高级语言 FORTRAN 以及相关的汇编程序和 FORTRAN 编译器，系统程序出现了。

1. 引入作业

为了运行以穿孔卡片形式出现的高级语言（汇编语言）源程序，操作员首先取来存有 FORTRAN 编译器的卡片以加载编译器，之后装载待运行程序卡片。然后对程序进行编译和链接，得到可运行程序。最后运行程序得到结果予以打印输出。这一组与运行一个程序有关的所有过程称为一个作业。

很显然，如果采用人工操作方式，在作业的执行过程中，绝大多数时间浪费在操作员在机房中走来走去安装卸载卡片，此时 CPU 无事可做。当时的计算机是极昂贵的，人们自然就想到了改进机器的体系结构将这一部分时间节省下来。

2. 批处理技术

为此，机房需要为中央计算机配备面向字符输入输出的低档计算机（简称输入/输出机）。系统工作方式如下：操作员收集一批待运行作业的卡片输入至输入机，该机器按顺序将全部卡片拷贝至输入磁带；中央计算机空闲时，操作员将输入磁带装入中央处理机，启动中央计算机运行输入磁带中的第一个作业。每当一个作业运行完毕，系统将其运行结果送输出磁带，

之后，从输入磁带中读取下一个作业。周而复始，直至输入磁带中的全部作业运行完毕；这样在一批作业全部执行完毕后输出磁带上将携带着所有作业的运行结果。操作员将其卸下，启动另一台输出机进行打印。

在中央计算机运行程序时，输入机可以输入下一组待执行作业，而输出机则可以输出上一组程序的运行结果。对中央计算机而言，以前浪费于操作员在机房中走来走去安装卸载卡片的时间被省下来了，运行效率有了很大的提高。这就是批处理技术。

批处理方式的体系结构需要软件的支持。首先，我们需要作业控制语言用于描述用户程序的特征。如代码 1.1 所示，为 FMS (FORTRAN Monitor System) 作业格式的一个实例。

```
$ JOB 最长运行时间, 计费账号, 程序员姓名
$ FORTRAN (或ASSEMBLER) ; 告诉系统下面的程序使用什么语言

..... ; 程序源代码

$ LOAD ; 将编译后的程序放入内存
$ RUN ; 用后面的数据运行程序

..... ; 程序运行时需要使用的数据

.....
$ END ; 作业到此结束
```

代码 1.1：作业控制语言实例

其次，系统需要一个常驻内存的监控程序，用于自动装载、运行存储在输入输出磁带中的每个作业。它的功能包括：

(1) 设备驱动：中央处理器执行完一个作业后，磁带驱动程序读入下一个作业，同时，向输出磁带送出作业的执行结果。

(2) 命令解释器：用于解释作业控制语言指令，并能够根据作业控制语言指令正确地对用户的作业进行处理。如读到 FORTRAN 指令，能够在系统程序磁带中找到 FORTRAN 程序的编译程序，装载之以对 FORTRAN 源程序进行编译。

(3) 对用户程序运行中的行为进行监控：在冯·诺伊曼结构的机器中，只有存储在内存中的程序才是可执行的。因此，用户程序在运行时将和监控程序共享主存。为了安全，监控程序将被放在内存中不允许用户程序访问的固定位置，时时监视用户程序有意无意访问监控程序的企图，阻止其对系统进行破坏。这是内存管理程序的雏形。

(4) 中断：计时器的工作需要中断机制的配合（计时结束后，计时器发出中断，以此激活监控程序），其它外设的工作情况也是如此；另外为了使外部设备始终处于正常的运行状态，用户程序是不允许直接使用外设的（系统不能指望用户程序不出错，一旦出错外设将不再工作，系统只有重新启动才能恢复它的工作）。当用户程序需要使用外设时，可以通过中断机制请求监控程序的服务。

(5) 时钟管理：为了防止一个作业无限制地使用计算机，系统在每个作业开始处理时启动计时器，如果作业在它申明的最大运行时间到来之前还没有运行完毕，将被系统强行终止。控制返回监控程序，执行下一个作业。

(6) 机器的工作状态和特权指令：系统充分相信监控程序的代码，认为它不会破坏系统；用户代码则不可信，需要系统对其进行监视。于是 CPU 的寄存器中引入了区分 CPU 正在执行指令可信程度的机器运行状态位。当 CPU 运行监控程序代码时，运行状态位标记机器为核心态（管态）运行。此种状态下，特权指令有效，机器可以访问内存中的所有单元。

否则，称机器运行在用户态下。在用户态下特权指令的执行将引起异常（一种特殊的中断）；正在运行指令访问的地址如果不属于存放该用户程序的那一部分内存也是一个异常事件。监控程序负责处理异常事件。

3. 单道批处理系统及其特征

早期的批处理系统中，在内存中始终只保持一道作业，故称其为单道批处理系统。图 1.2 示出了单道批处理系统的处理流程。上面介绍的监控程序可以算是单道批处理操作系统。其实严格地说，这里的监控程序只能算是操作系统的前身，而非现在人们所理解的操作系统。

尽管如此，单道批处理系统比起人工操作方式的系统已有很大进步。主要特征如下：

- (1) 自动性。在顺利情况下，在磁带上的一批作业能自动地、逐个地依次运行，而无需人工干预。
- (2) 顺序性。磁带上的各道作业是顺序进入内存的，各道作业的完成顺序与它们进入内存的顺序，在正常情况下应完全相同，亦即先调入内存的作业先完成。
- (3) 单道性。内存中仅有一道程序运行，即监控程序每次从磁带上只调入一道程序进入内存运行，当该程序完成或发生异常时，才换入其后续程序进入内存运行。

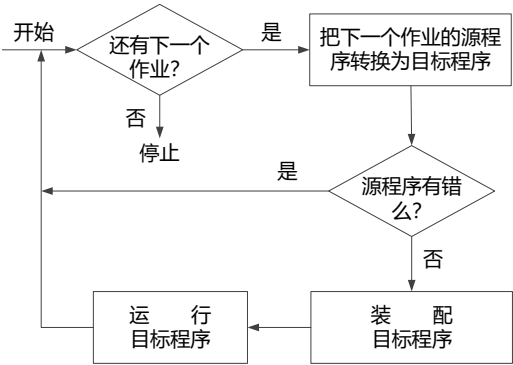


图 1.2：单道批处理系统地处理流程

1.2.3 多道批处理系统

20 世纪 60 年代中期，人们开始利用小规模集成电路来制造计算机，生产出第三代计算机，较之于晶体管计算机，无论在体积、功耗、速度和可靠性上，都有了显著的改善。高性能的计算机显然不能再采取单道批处理的方式了。因为如果内存中只有一道程序，无论它是科学运算程序还是商务数据处理程序，在运行中都会造成计算机资源的极大浪费。我们知道，科学计算涉及很少的 I/O 操作。这类程序运行时，CPU 非常忙碌，I/O 设备几乎无事可做；而进行商务数据处理时，I/O 操作较频繁，在有的商务数据处理程序中，CPU 用于等待 I/O 设备读写文件的时间竟然高达 95% 以上。怎样才能充分利用昂贵的计算机资源，使 CPU 和外设同时工作起来。这个强烈的需求直接导致了计算机多道程序概念的诞生。操作系统的理论和实践由此进入了一个飞速发展的时期。

1. 多道程序设计的基本思想

简单地说，多道程序设计技术的基本思想是：用户所提交的作业都先存放在外存上并排成一个队列，称为“后备队列”；然后，由作业调度程序按一定的算法从后备队列中选择若干个作业调入内存，使它们共享 CPU 和系统中的各种资源。当一个程序启动外设等待 I/O 操作时，处理机可以运行另一个需要使用 CPU 的程序。这样就能使 CPU 和 I/O 操作并行执行，最大限度地提高计算机各种资源（不止是 CPU）的利用率。

多道程序的运行具备如下特点：

- (1) 多道：计算机内存中同时存放多道相互独立的程序。
- (2) 宏观上并发执行：同时进入系统的几道程序都处于运行状态，但都未运行结束。
- (3) 微观上串行运行：各作业交替使用 CPU。

在操作系统中引入多道程序设计技术可以带来以下好处：

(1) 提高 CPU 的利用率。当内存中仅有一道程序时，每逢该程序在运行中发出 I/O 请求后，CPU 即空闲，必须在其 I/O 完成后 CPU 才能继续运行。由于 I/O 设备的低速性，使得 CPU 的利用率大大降低。图 1.3 (a) 示出了单道程序的运行情况，从图中可以看出：在 $t_2 \sim t_3$ 、 $t_6 \sim t_7$ 时间间隔内 CPU 空闲。在引入多道程序设计技术后，由于同时在内存中装有若干道程序，并使得它们交替地运行，这样，当正在运行的程序因 I/O 而暂停执行时，系统可调度另一道程序运行，从而保持了 CPU 处于忙碌状态。如图 1.3 (b) 所示为四道程序时的运行情况。

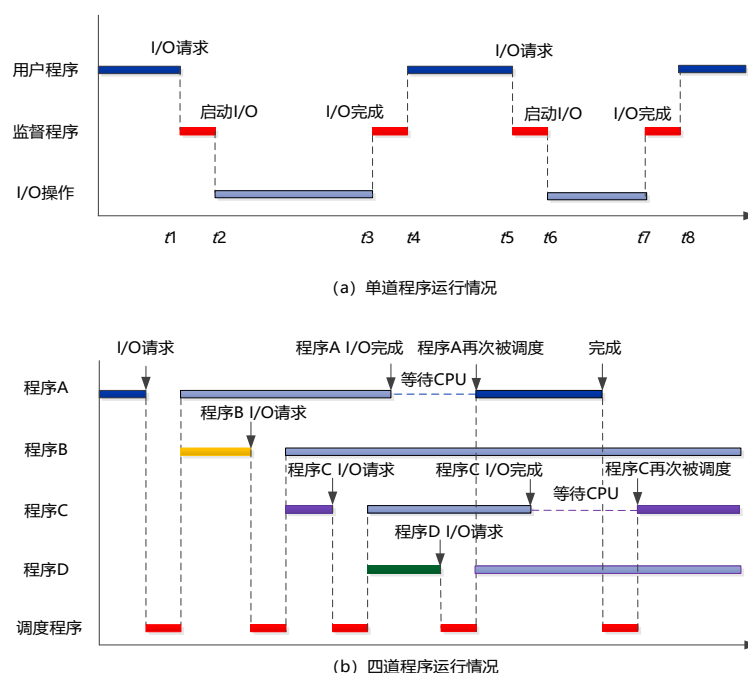


图 1.3：单道和多道程序运行情况

(2) 提高内存和 I/O 设备利用率。为了能运行较大的作业，通常内存都具有较大容量，但由于 80% 以上的作业都属于中小型，因此在单道程序环境下，必定造成内存的浪费。类似地，对于系统中所配置的多种类型的 I/O 设备，在单道程序环境下也不能充分利用。如果允许在内存中装入多道程序，并允许它们并发执行，则无疑会大大提高内存和 I/O 设备的利用率。

(3) 增加系统吞吐量。在保持 CPU、I/O 设备不断忙碌的同时，也必然会大幅度地提高系统的吞吐量，从而降低作业运行所需的费用。

多道程序设计技术具有上述优势，但是也带来了新的技术问题，包括：多道程序的并发运行需要共享软硬件资源，需要同步和互斥机制；多道程序需要提高内存的使用效率，需要覆盖技术、交互技术、虚拟存储等技术；多道程序在内存中要保证系统存储区和用户存储区安全可靠，需要可靠的内存保护机制。这些相关技术都将在后续章节中陆续介绍。

2. 多道批处理系统

在批处理中采用多道程序设计技术，就形成了多道批处理系统。虽然早在上世纪 60 年代就已经出现了多道批处理系统，但至今它仍是三大基本操作系统类型之一，说明它具备了其他类型操作系统所不具有的优点，例如：资源利用率高，系统吞吐量大等。

在多道批处理系统中，作业从进入到退出系统大致分为四个阶段（见图 1.4）。

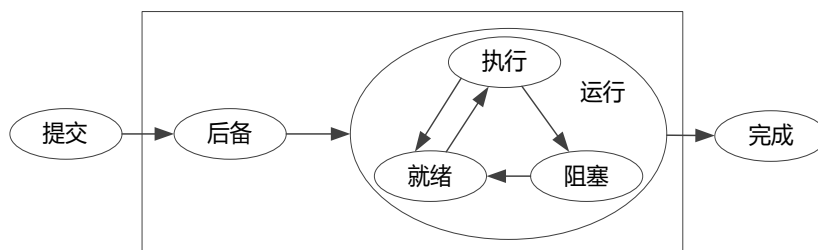


图 1.4: 多道批处理系统中作业的状态转换

(1) 用户准备好作业程序、数据以及说明如何运行该作业的操作说明书，然后将它们提交给系统。早期系统中，作业的准备和提交的方式主要是将它们穿成卡片或纸带，并送交机房，由操作员使用纸带或卡片输入机将作业提交给系统。现代的多道批处理系统中，主要是由用户使用文本编辑器将作业构成文件存放在文件系统中，使用终端直接向系统提交作业。此时的作业处于提交状态。

(2) 作业提交后，系统将它存放在磁盘的某些盘区中等待运行。这些盘区称为输入井，其中可以同时存放许多等待运行的作业。这些作业称为后备作业，处于后备状态。

(3) 系统从等待运行的后备作业中，挑选出若干作业，将它们送入内存，并按一定方式使它们在处理机上运行。

挑选后备作业时，一般要考虑到尽可能多得使用系统内包含的各种资源。在内存中的几道作业程序，任一时刻只能有一道（在单处理机环境下）或几道（在多处理环境下）在处理机上执行。当一个作业因某种原因（例如，等待输入或输出操作结束）不能继续执行时，就放弃使用处理机，暂时被阻塞以等待该原因消失。此时，系统安排处理机执行另一道准备就绪的作业程序。如此反复，使得各作业程序交替地在处理机上运行。需要强调的是，此时，在内存中的各道作业程序都已经开始运行，但都尚未结束，因此可以从宏观上认为它们都在运行，处于运行状态。

(4) 作业运行结束后，系统收回该作业占用的资源，并使其退出系统。此时作业处于完成状态。一个作业运行结束后，系统根据当时资源的使用情况以及各后备作业的特点，重新选择一个或几个作业送入内存，并开始它们的运行过程。

多道程序批处理系统必须依赖于某些计算机硬件功能，它们是：

(1) 外设中断：CPU 和 I/O 操作在中断控制器的作用下可以并行执行。在 CPU 发出 I/O 命令后，正在运行的程序 A 一般需要等待外设 I/O 操作结束后才能够继续运行下去，为了不浪费 CPU 的运算能力，程序 A 将主动放弃 CPU 的使用权。系统选择内存中的另一个程序运行。当 I/O 操作结束后，由外设控制器向 CPU 发中断请求信号，CPU 响应中断，暂停当前程序 B 的运行，执行中断处理程序，在执行中断处理程序的过程中以某种方式通知程序 A，I/O 操作已结束可以继续运行下去了。服务结束后，立即恢复程序 B 的运行（或执行其它程序）。

(2) DMA 技术：中断技术不适合操纵硬盘等块设备。因为这些设备传送一个字节所需的时间极短来不及运行中断处理程序，而硬盘永不停息的匀速圆周运动的工作方式使其不可能在一个字节传送完毕后停下来等待中断处理程序运行完毕。为了操纵硬盘，系统需要一个支持直接存储器访问方式的硬件——DMA 控制器。在它的控制下，来自硬盘的数据能够通过 CPU 中的寄存器直接写入内存（或相反），只在整块数据传送的开始和结束时才需要 CPU 的干预（鉴于此，该方式又被称为成组数据传输方式）。DMA 的出现对计算机的体系结构产生了重要的影响，它使 CPU 和 I/O 设备能够共享主存，并最终使主存取代了处理机在系统中的中心地位。

同时，多道程序批处理系统的实现需要操作系统代码进行大规模地改进，以管理共享内存的多道程序。它们是：

(1) 存储管理子系统：内存中的用户区为多道程序共享，系统必须引进大量的数据结构和代码用于登记和管理每个程序在内存中存放的位置以确保用户程序不会相互干扰（需要硬件的配合）；此外还需要对尚未分配出去的内存空间进行登记和管理。

(2) 处理机管理子系统：内存中驻留多个应用程序，经常会发生不止一个程序具备运行条件（该程序没有启动 I/O 操作）的情况。此时 CPU 应该分配给哪个程序？对这个问题的思考导致处理机调度算法的产生和发展。

多道批处理系统也有一定的缺陷，主要表现在：

(1) 平均周转时间长。作业的周转时间是指从作业进入系统开始，直至其完成并退出系统为止所经历的时间。在批处理系统中，由于作业要排队，依次进行处理，因而作业的周转时间较长，通常需要几个小时，甚至几天。

(2) 无交互能力。用户一旦把作业提交给系统后，直至作业完成，用户都不能与自己的作业进行交互，这对修改和调试程序是极不方便的。

1.2.4 分时系统

1. 分时系统的产生

上面所描述的多道批处理系统，完全从系统的角度出发，极大地提高了系统中各种资源的利用率。但用户提交作业后，仍然要等到几个小时后才能得到运行的结果。程序员尤其不喜欢这种工作方式。他们喜欢一个人独占机器一段时间以便快速查出程序中的错误，而不是在编译通过后两个小时才被告知源程序某处少了一个逗号。更为重要的是，商务数据处理需要用用户直接与计算机交互，批处理的工作方式根本不适合这类应用。这种需求最终导致分时多道操作系统的出现。

或者说，分时系统是为了满足用户需求所形成的一种新型操作系统。它与多道批处理系统之间，有着截然不同的性能差别。用户的需求具体表现在以下几个方面：

(1) 人机交互。每当程序员写好一个新的程序时，都需要上机进行调试。由于新编程序难免有错误之处需要修改，因而希望能够像早期使用计算机时一样对它进行直接控制，并能以一边运行一边修改的方式，对程序中的错误进行修改。

(2) 共享主机。在 20 世纪 60 年代，计算机非常昂贵，不可能像现在这样每人独占一台微机，而只能是多个用户共享一台计算机，但用户在使用计算机时应能够像自己独占计算机一样，不仅可以随时和计算机交流，而且应感觉不到其他用户也在使用该计算机。

(3) 便于用户上机。在多道批处理系统中，用户上机前必须把自己的作业送到机房，这对于用户尤其是远地用户来说是十分不便的。用户希望能够通过自己的终端直接将作业传送到机器上进行处理，并能对自己的作业进行控制。

所谓分时系统是指，在一台主机上连接了多个带有显示器和键盘的在线终端，每个上机用户拥有一个终端，同时允许多个用户，以交互方式使用计算机，共享主机中的资源。用户在键盘上输入的命令行通过通信线路送达主机，程序的运行结果同样通过通信线路送回终端的显示屏。在一个有 20 个用户同时上机的分时系统中，如果 17 个用户正在思考问题，CPU 将轮流分配给需要服务的另外 3 个用户。很明显，如果有 N 个用户同时请求服务，在不计操作系统开销的情况下，每个用户将得到计算机有效速度的 $1/N$ 。但相对计算机而言人的反应速度是很慢的，所以设计良好的系统，其响应速度是能够令用户满意的。多用户分时操作系统是当今计算机操作系统中使用最普遍的一类操作系统。本书介绍的 UNIX 就是多道分时操作系统的典范。

2. 分时系统的工作方式

一个分时计算机系统往往带有几十台甚至上百台终端设备, 每个用户在他所占用的终端上控制他的作业运行, 所以分时系统也称为多路存取系统。分时系统的基本工作过程如下:

- (1) 每个用户使用终端向系统发出命令, 要求系统完成某项工作;
- (2) 系统对从终端设备上发来的命令进行分析, 然后执行该命令, 最后将执行结果通过相应终端的显示部分回送给用户;
- (3) 用户根据上一条命令的执行结果, 向系统发出下一条命令;
- (4) 不断重复上述会话过程, 直到完成用户预定的工作为止。

在分时系统中, 用户可以与运行中的作业交互, 在线输入数据、及时获得输出, 甚至还能对它实施控制, 暂停或终止其执行过程。

3. 分时系统的基本特征

需要强调的是, 这里分时的意思是指: 把处理机的运行时间分成很短的时间段, 每个时间段称为一个时间片, 系统按时间片轮流把处理机分配给各个作业使用。如果某个作业在分配给它的时间片结束时还未完成, 则强迫该作业放弃使用处理机, 等待下一轮循环再继续运行。此时, 系统将处理机分配给另一作业使用。当然, 作业在运行中一旦要求进行输入、输出操作, 也自动放弃处理机, 待相应操作结束后, 再等待分配处理机。由于终端上用户的工作速度相对于机器而言都是很慢的, 所以系统一般都能在一个时间片内轮转响应所有用户的请求。由于各个用户的请求都能得到较快的响应, 因此给每个用户造成一种印象, 好像他独占该计算机一样。

4. 分时系统实现中的关键问题

分时系统中最关键的问题是, 如何让用户与自己的作业进行交互, 即: 当用户在自己的终端上键入命令时, 系统应能及时接收并处理该命令, 再将结果返回给用户。此即人机交互。应强调指出, 即使有多个用户同时通过自己的键盘键入命令, 系统也应能全部地及时接收并处理这些命令。

(1) 及时接收。要及时接收用户键入的命令或数据并不困难, 为此, 只需在系统中配置一个多路卡。例如, 当要在主机上连接 8 个终端时, 需配置一个 8 用户的多路卡。多路卡的作用是使得主机能同时接收各用户从终端上输入的数据。此外, 还须为每个终端配置一个缓冲区, 用来暂存用户键入的命令 (或数据)。

(2) 及时处理。人机交互的关键, 是让用户键入命令后能及时地控制自己作业的运行, 或修改自己的作业。为此, 各个用户的作业都必须在内存中, 且应能频繁地获得处理机而运行; 否则, 用户键入的命令将无法作用在自己的作业上。前面介绍的批处理系统是无法实现人机交互的, 因为通常大多数作业都还驻留在外存上, 即使是已调入内存的作业也经常要经过较长时间的等待后方能运行, 因而使得用户键入的命令很难及时作用到自己的作业上。

由此可见, 为实现人机交互, 必须彻底改变原来批处理系统的运行方式。首先, 用户作业不能先进入磁盘, 然后再调入内存。因为作业在磁盘上不能运行, 当然用户也无法与机器交互。因此, 作业应能直接进入内存。其次, 不允许一个作业长时间占用处理机, 直至它运行结束或出现 I/O 请求后, 方才调度其他作业运行。为此, 应该规定每个作业只能运行一个很短的时间片, 然后便停止该作业的运行, 并立即调度下一个程序运行。如果在不长的时间内, 能让所有的用户作业都执行一次, 便可实现用户与自己作业的交互, 从而可使用户的请求得到及时响应。

1.2.5 实时系统

所谓“实时”，是指系统能及时响应外部事件的请求，在规定的时间内完成对该事件的处理，并控制所有实时任务协调一致地运行。

1. 实时系统的产生

虽然多道批处理系统和分时系统已能获得较为令人满意的资源利用率和响应时间，从而使计算机的应用范围日益扩大，但它们仍然不能满足以下某些应用领域的需要。

(1) 实时控制。当把计算机用于生产过程的控制、武器的控制，以形成以计算机为中心的控制系统时，要求系统能够实时采集现场数据，并对数据进行及时处理，进而自动地控制相应的执行机构。通常把这些用于实时控制的系统称为实时系统。

(2) 实时信息处理。通常，人们把用于对信息进行实时处理的系统称为实时信息处理系统。该系统由一台或多台主机通过通信线路连接到成百上千个远程终端上，计算机接收从远程终端上发来的服务请求，根据用户提出的请求对信息进行检索和处理，并在很短的时间内为用户做出正确的响应。典型的实时信息处理系统有飞机或火车的订票系统、情报检索系统等。

2. 实时系统的特征

在实时系统中必然存在着若干个实时任务，这些任务通常与某些外部设备相关，能反应或控制相应的外部设备，因而带有某种程度的紧迫性。其表现出的基本特征是：及时性要求高、系统可靠性高，需要实时的时钟管理及过载保护。

3. 实时系统与分时系统的比较

实时系统有着与分时系统相似但并不完全相同的特点，表 1.1 从五个方面对这两种系统进行了比较。

表 1.1：实时系统和分时系统的比较

特点	实时系统	分时系统
多路性	实时控制系统中体现在周期性地对多路现场信息进行采集、对多个对象或多个执行机构进行控制。实时信息处理系统也按分时原则为多个终端用户服务。	按分时原则为多个终端用户服务。多路性与用户情况有关，时多时少。
独立性	每个终端用户向系统提出服务请求时，彼此独立操作，互不干扰；对信息的采集和对象的控制也彼此互不干扰。	每个用户各占一个终端，彼此互不干扰，独立操作。
(续表)		
特点	实时系统	分时系统
及时性	以控制对象所要求的开始截止时间或完成截止时间来确定，一般为秒级或毫秒级。	用户的请求能在短时间（用户能接受的）内获得响应。
交互性	人与系统的交互，仅限于访问系统中某些特定的专用服务程序。	用户与系统进行广泛的人机对话，系统能向终端用户提供数据处理，资源共享等服务。
可靠性	高度可靠	可靠程度较低

1.2.6 操作系统的进一步发展

操作系统经过 20 世纪 60 年代、70 年代的发展，到 20 世纪 80 年代已趋于成熟，随着大规模集成电路和计算机体系结构的发展，逐渐形成了微机操作系统、多处理机操作系统、

网络操作系统、分布式操作系统和嵌入式操作系统等。这里我们将扼要介绍微机操作系统的发展。

配置在微机上的操作系统称为微机操作系统。微机操作系统可按微机字长分成 8 位、16 位、32 位、64 位。但也可把微机操作系统按其运行方式的不同，分为单用户单任务操作系统、单用户多任务操作系统。

1. 单用户单任务操作系统

单用户单任务操作系统的含义是：只允许一个用户上机，且只允许用户程序作为一个任务运行。这是最简单的操作系统，主要配置在 8 位和 16 位微机上。最具代表性有 CP/M 和 MS-DOS。CP/M 是在 1975 年由 Digital Research 公司率先推出的 8 位微机操作系统，具有较好的层次结构、可适应性、可移植性及易学易用性，在 8 位微机操作系统中占据了统治地位。MS-DOS 在 CP/M 的基础上进行了较大的扩充，增加了许多内、外部命令，使该操作系统具有较强的功能及性能优良的文件系统。随着 IBM-PC 机及其兼容机的畅销，MS-DOS 成为 16 位单用户单任务操作系统的标准。

2. 单用户多任务操作系统

单用户多任务操作系统的含义是：只允许一个用户上机，但允许将一个用户程序分为若干个任务并发执行，从而有效地改善系统的性能。在 32 位微机上配置的操作系统基本上都是单用户多任务操作系统，其中最具代表性的是 WINDOWS，它是微软公司提供的一种支持图形用户界面（GUI）方式的操作系统，包括：1990 年 5 月推出的 WINDOWS 3.0，1993 年推出的 WINDOWS NT，以及之后又陆续推出的 WINDOWS 95/ WINDOWS 98/2000/XP。

3. 多用户多任务操作系统

多用户多任务操作系统的含义是：允许多个用户通过各自的终端使用同一台机器，共享主机系统中的各种资源，而每个用户程序又可进一步分为几个任务，使它们能够并发执行。在大、中、小型机中所配置的大多数是多用户多任务操作系统，其中最具有代表性的是 UNIX。UNIX 是美国电报电话公司的 Bell 实验室在 1969~1970 年期间开发的，1979 年推出的 UNIX V7 已被广泛应用于多种中、小型机器上。随着微机性能的提高，人们又将 UNIX 移植到微机上。现在最有影响的两个能在微机上运行的 UNIX 的变型是 Linux 和 Solaris。

1.3. 操作系统的基本特征

前述的三种基本操作系统都有自己的特征，如批处理系统具有能对多个作业进行成批处理，以获得高的系统吞吐量的特征，分时系统具有允许用户和计算机进行人机交互的特征，而实时系统具有实时特征，但它们也都具有并发、共享、虚拟和异步这四个现代操作系统的基本特征。其中，并发特征是操作系统最重要的特征，其他三个特征都是以并发特征为基础的。

1.3.1 并发性

1. 并行与并发

并行性和并发性是既相似又有区别的两个概念。并行性是指两个或多个事件在同一时刻发生；而并发性是指两个或多个事件在同一时间间隔内发生。在多道程序环境下，并发性是指在一段时间内宏观上有多个程序在同时运行，但在单处理机系统中，每一个时刻仅能有一道程序执行，故微观上这些程序只能是分时地交替执行。倘若在计算机系统中有多个处理器，

则这些可以并发执行的程序便可被分配到多个处理机上,实现并行执行,即利用每个处理器来执行一个可并发执行的程序,这样,多个程序便可以同时执行。

程序的并发执行,有效地改善了系统资源的利用率,提高了系统的吞吐量,但它使系统复杂化,操作系统必须具有控制和管理各种并发活动的的能力。

2. 进程概念的引入

在多道程序的环境中,一个程序在运行过程中会随时受到内存中其它程序的干扰,不再可能一步运行到底。因此系统需要在程序暂停时保留其全部运行状态,以便恢复运行时能够完全继承以前的运行结果。程序将不再是静态的代码和数据,与之相联系的还应该有运行状态。于是,进程的概念出现了,它用于刻画运行中的程序。

实际上,引入进程的好处还远不止于此。内存中可以存在多个用户程序,分别为它们建立进程后,这些进程便可以并发执行,亦即实现前面所说的多道程序运行。

程序的并发执行将使系统复杂化,以至于在系统中必须增加新的功能模块,分别对于处理机、内存、I/O 设备和文件系统等资源进行管理,并控制系统中作业的运行。事实上,进程和并发是现代操作系统中最重要的基本概念,也是操作系统运行的基础。我们将在第二章中做详细介绍。

1.3.2 共享性

在操作系统环境下,所谓共享,是指系统中的资源可供内存中多个并发执行的进程同时使用。由于各种资源的属性不同,进程对资源共享的方式也不同,目前主要实现资源共享的方式有以下两种。

1. 互斥访问方式

系统中的某些资源,如打印机、磁带机,虽然它们可以提供给多个进程使用,但为使所打印或记录的结果不致造成混淆,应规定在一段时间内只允许一个进程访问该资源。为此,系统中应建立一定的机制,以确保对这类资源的互斥访问。例如,当某一进程需要使用某个资源时,必须先提出请求,如果此时该资源空闲,系统便可将之分配给请求进程;反之,若该资源正被占用,则请求进程必须等待。

我们把这种资源共享方式称为互斥式共享,而把在一段时间内只允许一个进程访问的资源称为临界资源或独占资源。计算机系统的大多数物理设备,以及某些软件中所用的栈、变量和表格,都属于临界资源,它们要求被互斥地共享。

2. 同时访问方式

系统中还有另一类资源,允许在一段时间内由多个进程“同时”对它们进行访问。这里所谓的“同时”往往是宏观上的,而在微观上,这些进程可能是交替地对该资源进行访问。典型的可供多个进程“同时”访问的资源是磁盘设备,一些用重入码编写的文件,也可以被“同时”共享,即若干个用户同时访问该文件。

并发和共享是操作系统两个最基本的特征,这两者之间又是互为存在条件的。一方面,资源共享是以进程的并发为条件的,若系统不允许程序并发执行,自然不存在资源共享问题。另一方面,若系统不能对资源共享实施有效的管理,也必将影响到程序的并发执行,甚至根本无法并发执行。

1.3.3 虚拟技术

在操作系统中,所谓虚拟,指的是把一个物理上的实体,变为若干个逻辑上的对应物。物理实体(前者)是实的,即实际存在的;而后者是虚的,是用户感觉上的东西。相应地,

用于实现虚拟的技术，称为虚拟技术。在操作系统中利用了多种虚拟技术，分别用来实现虚拟处理机、虚拟内存、虚拟外部设备等。

1. 虚拟处理机技术

在虚拟处理机技术中，通过多道程序设计，为每道程序建立一个进程，用多道程序并发执行，来分时使用一台处理机。此时，虽然只有一台处理机，但它却能同时为多个用户服务，使每个终端用户都认为是有一个 CPU 在专门为他服务。亦即，利用多道程序设计技术，把一台物理上的 CPU 虚拟为多台逻辑上的 CPU，在每台逻辑 CPU 上运行一道程序。这样的逻辑 CPU 称为虚拟处理机。

2. 虚拟存储器技术

类似地，可以通过虚拟存储器技术，将一台机器的物理存储器变为虚拟存储器，以便从逻辑上扩充存储器的容量。此时，虽然物理内存的容量可能不大（如 32MB），但它可以运行比它大得多的用户程序（如 128MB）。这使用户所感觉到的内存容量比实际内存容量大得多，认为该机器的内存至少也有 128 MB。当然这时用户所感觉到的内存容量是虚的。我们把用户所感觉到的存储器称为虚拟存储器。

3. 虚拟设备技术

我们还可以通过虚拟设备技术，将一台物理上的 I/O 设备虚拟为多台逻辑上的 I/O 设备，并允许每个用户占用一台逻辑上的 I/O 设备，这样便可使原来仅允许在一段时间内由一个用户访问的独占设备，变为在一段时间内允许多个用户同时访问的共享设备。例如，原来的打印机属于独占设备，只能为一个用户程序服务，而通过虚拟设备技术，可以把它变为多台逻辑上的打印机，供多个用户“同时”打印。

1.3.4 异步性

在多道程序环境下，允许多个进程并发执行，但只有进程在获得所需的资源后方能执行。在单处理机环境下，由于系统中只有一个处理机，因而每次只允许一个进程执行，其余进程只能等待。当正在执行的进程提出某种资源要求时，如打印请求，如果此时打印机正在为其它进程打印，由于打印机属于临界资源，因此正在执行的进程必须等待，且放弃处理机，直到打印机空闲，并再次把处理机分配给该进程时，该进程方能继续执行。可见，由于资源等因素的限制，使进程的执行通常都不是“一气呵成”，而是以“停停走走”的方式运行。

内存中的每个进程在何时能获得处理机运行，何时又因提出某种资源请求而暂停，以及进程以怎样的速度向前推进，每道程序总共需多少时间才能完成，等等，都是不可预知的。由于各用户程序性能的不同，比如，有的侧重于计算而较少需要 I/O，而又有的程序其计算少而 I/O 多，这样，很可能是先进入内存的作业后完成，而后进入内存的作业先完成。或者说，进程是以人们不可预知的速度向前推进，此即进程的异步性。

尽管如此，只要运行环境相同，作业经多次运行，都会获得完全相同的结果。因此，异步运行方式是允许的，是操作系统的一个重要特征。

1.4. 操作系统的基本功能

操作系统的主要任务，是为多道程序的运行提供良好的运行环境，以保证多道程序能

有条不紊地、高效地运行,并能最大程度地提高系统中各种资源的利用率和方便用户的使用。为实现上述任务,操作系统应具有这样几方面的功能:进程管理、存储管理、设备管理和文件管理。此外,为了方便用户使用操作系统,还须提供方便的用户接口。

1.4.1 进程管理功能

进程管理的主要功能是创建和撤销进程,对诸进程的运行进行协调,实现进程之间的信息交换,以及按照一定的算法把处理机及系统内的其他资源分配给进程。

1. 进程控制

在多道程序环境下,要使作业运行,必须先为它创建一个或几个进程,并为之分配必要的资源。当进程运行结束时,立即撤销该进程,以便能及时回收该进程占用的各类资源。进程控制的主要功能是为作业创建进程、撤销已结束的进程,以及控制进程在运行过程中的状态转换。

2. 进程同步

前已述及,进程是以异步方式运行的,并以人们不可预知的速度向前推进。为使多个进程能有条不紊地运行,系统中必须设置进程同步机制。进程同步的主要任务是协调多个进程的运行。协调方式有两种,包括:(1)进程互斥方式,这是指诸进程在对临界资源进行访问时,应采用互斥方式;(2)进程同步方式,指在相互合作去完成共同任务的诸进程间,由同步机构对它们的执行次序加以协调。

3. 进程通信

在多道程序环境下,由于多个进程共享系统资源,且可能存在多个进程相互合作去完成一个共同的任务。而在这些进程之间,又往往需要交换信息。进程通信的任务就是用来保证如果有效的在进程之间进行信息交换。

4. 调度

在后备队列上等待的每个作业,通常都要经过调度才能执行。在传统的操作系统中,包括作业调度和进程调度两步。作业调度的基本任务,是按照一定的算法,从后备队列中选择出若干个作业,为它们分配其必需的资源(首先是分配内存)。在它们调入内存后,便分别为它们建立进程,使它们都成为可能获得处理机的就绪进程,并按照一定的算法将它们插入就绪队列。而进程调度的任务,则是从进程的就绪队列中选出一新进程,把处理机分配给它,并为它设置运行现场,使进程投入执行。

1.4.2 存储管理功能

存储管理的主要任务是为多道程序的运行提供良好的环境,方便用户使用存储器,提高存取器的利用率以及能从逻辑上扩充内存。为此,存储管理应具有内存分配、内存保护、地址映射和内存扩充等功能。

1. 内存分配

内存分配的主要任务是为每道程序分配内存空间,使它们“各得其所”;提高存储器利用率,以减少空间浪费;允许正在运行的程序申请附加的内存空间,以适应程序和数据动态增长的需要。

为了实现内存分配,内存分配机制应具有一套内存分配数据结构,该结构用于记录内存空间的占用情况,作为内存分配的依据;其次,能够按照一定的内存分配算法,为用户程序

分配内存空间；系统对于用户不再需要的内存，通过用户的释放请求，去完成系统的回收功能。

2. 内存保护

内存保护的主要任务，是确保每道用户程序都只在自己的内存空间内运行，彼此互不干扰。决不允许用户程序访问操作系统的程序和数据；也不允许用户程序转移到非共享的其他用户程序中去执行。

为了确保每道程序都只在自己的内存区中运行，必须设置内存保护机制。一种比较简单的内存保护机制，是设置两个界限寄存器，分别用于存放正在执行程序的上界和下界。系统须对每条指令所要访问的地址进行检查，如果发生越界，便发出越界中断请求，以停止该程序的执行。如果这种检查完全用软件实现，则每执行一条指令，便须增加若干条指令去进行越界检查，这将显著降低程序的运行速度。因此，越界检查都由硬件实现。当然，对发生越界后的处理，还须与软件配合来完成。

3. 地址映射

一个应用程序（源程序）经编译后，通常会形成若干个目标程序；这些目标程序再经过链接便形成了可执行程序。这些可执行程序的起始地址都是从“0”开始的，程序中的其它地址都是相对于起始地址计算的；由这些地址所形成的地址范围称为“地址空间”，其中的地址称为“逻辑地址”或“相对地址”。与此相应的是，由计算机系统的物理内存中的一系列单元所限定的地址范围称为“内存空间”，其中的地址称为“物理地址”。

可执行程序在执行前需被装入内存。在多道程序环境下，每道程序不可能都从“0”地址开始装入，这就致使地址空间内的逻辑地址和内存空间中的物理地址不相一致。为使程序能正确运行，存储管理必须提供地址映射功能，以将地址空间中的逻辑地址转换为内存空间中与之对应的物理地址。为保证程序执行的效率，该功能应在硬件的支持下完成。

4. 内存扩充

存储器管理中的内存扩充任务，并非是去扩大物理内存的容量，而是借助虚拟存储技术，从逻辑上扩充内存容量，使用户所感觉到的内存容量比实际内存容量大得多；或者是让更多的用户程序能并发运行。这样，既满足了用户的需要，改善了系统的性能，又基本上不增加硬件投资。

1.4.3 设备管理功能

设备管理用于管理计算机系统中所有的外围设备，而设备管理的主要任务是：完成用户进程提出的 I/O 请求；为用户进程分配其所需的 I/O 设备；提高 CPU 和 I/O 设备的利用率；提高 I/O 速度；方便用户使用 I/O 设备。为实现上述任务，设备管理应具有缓冲管理、设备分配和设备处理等功能。

1. 缓冲管理

CPU 运行的高速性和 I/O 设备低速性间的矛盾自计算机诞生时起便已存在。而且随着 CPU 速度的迅速提高，此矛盾日益突出，严重降低了 CPU 的利用率。而如果在 I/O 设备和 CPU 之间引入缓冲，则可有效地缓和这一矛盾，进而提高系统吞吐量。因此，在现代计算机系统中，都毫无例外地在内存中设置了缓冲区，而且还可通过增加缓冲区容量的方法，来改善系统的性能。

2. 设备分配

设备分配的基本任务是根据用户进程的 I/O 请求、系统的现有资源使用情况以及按照某种设备分配策略，为之分配其所需的设备。如果在 I/O 设备和 CPU 之间，还存在着设备控制器和 I/O 通道时，还须为分配出去的设备分配相应的控制器和通道。

为了实现设备分配，系统中应设置设备控制表、控制器控制表等数据结构，用于记录设备及控制器的标识符和状态。根据这些表格可以了解指定设备当前是否可用，是否忙碌，以供设备分配时参考。在进行设备分配时，应针对不同的设备类型采用不同的设备分配策略。对于独占设备（临界资源）的分配，还应考虑到该设备被分配出去后，系统是否安全。设备使用完后，还应立即由系统回收。

3. 设备处理

设备处理程序又称为设备驱动程序。其基本任务是实现 CPU 和设备控制器之间的通信，一方面，由 CPU 向设备控制器发出 I/O 命令，要求它完成指定的 I/O 操作；另一方面，由 CPU 接收从控制器发来的中断请求，并给予迅速的响应和相应的处理。

1.4.4 文件管理功能

在现代计算机中，总是把程序和数据以文件的形式存储在磁盘或磁带上，供所有的或指定的用户使用。为此，在操作系统中必须配置文件管理机构。文件管理的主要任务是对用户文件和系统文件进行管理，以方便用户使用，并保证文件的安全。为此，文件管理应具有对文件存储空间的管理、目录管理、文件的读/写管理，以及文件的共享和保护等功能。

1. 文件存储空间的管理

由文件系统对诸多文件及文件的存储空间，实施统一的管理。其主要任务是为每个文件分配必要的外存空间，提高外存的利用率，并能有助于提高文件系统的运行速度。为此，系统应设置相应的数据结构，记录文件存储空间的使用情况，供分配存储空间时参考；系统还应具有对存储空间进行分配和回收的功能。

2. 目录管理

为了使用户能方便地在外存上找到自己所需的文件，通常由系统为每个文件建立一个目录项。目录项包括文件名、文件属性、文件在磁盘上的物理位置等。目录管理的主要任务，是为每个文件建立有效的目录项，并对众多的目录项加以有效地组织，以实现方便的按名存取，即：用户只须提供文件名，即可对该文件进行存取。其次，应能提供快速的目录查询手段，提高文件检索速度。最后，目录管理还应能方便地实现文件共享，这样，只须在外存上保留一份该共享文件。

3. 文件的读写控制

在进行文件读（写）时，系统先根据用户给出的文件名，去检索文件目录，从中获得文件在外存中的位置。然后，利用文件读（写）指针，对文件进行读（写）。一旦读（写）完成，便修改读（写）指针，为下一次读（写）做好准备。由于用户对文件的读和写操作不会同时进行，故可合用一个读（写）指针。

4. 文件保护

为了防止系统中的文件被非法窃取和破坏，在文件系统中必须提供有效的存取控制手段，以实现下述目标：

- (1) 防止未经核准的用户存取文件；
- (2) 防止冒名顶替存取文件；

(3) 防止以不正确的方式使用文件。

1.4.5 操作系统与用户之间的接口

为了方便用户使用，操作系统必须向用户提供“用户与操作系统的接口”。该接口通常可分为两大类：

(1) 命令接口：它是提供给用户使用的接口，用户可通过该接口取得操作系统的服务；

(2) 程序接口：它是提供给程序员在编写程序时使用的接口，是用户程序取得操作系统服务的唯一途径。

1. 命令接口

为了便于用户直接或间接地控制自己的作业，操作系统向用户提供了命令接口。用户可通过该接口向作业发出命令以控制作业的运行。该接口又可进一步分为联机用户接口和脱机用户接口。

(1) 联机用户接口。这是为联机用户提供的，它由一组键盘操作命令及命令解释程序组成。当用户在终端或控制台上每键入一条命令后，系统便立即转入命令解释程序，对该命令加以解释并执行该命令。在完成指定功能后，控制又返回到终端或控制台上，等待用户键入下一条命令。这样，用户可通过先后键入不同命令的方式，来实现对作业的控制，直至作业完成。

用户虽然可以通过联机用户接口来取得操作系统的服务，但这时要求用户能熟记各种命令的名字和格式，并严格按照规定的格式输入命令。这样做既不方便又浪费时间，于是图形用户接口便应运而生。用户可以通过鼠标、菜单或对话框来完成对应用程序和文件的操作。

(2) 脱机用户接口。该接口是为批处理作业用户提供的，故也称为批处理用户接口。该接口由一组作业控制语言组成。批处理作业的用户不能直接与自己的作业交互作用，只能委托系统代替用户对作业进行控制和干预。这里的作业控制语言便是提供给批处理作业用户的、为实现所需功能而委托系统代为控制的一种语言。用户用该语言把需要对作业进行的控制和干预，事先写在作业说明书上，然后将作业连同作业说明书一起提供给系统。当系统调度到该作业运行时，同时调用命令解释程序，对作业说明书上的命令，逐条地解释执行。如果作业在执行过程中出现异常现象，系统也将根据作业说明书上的指示进行干预。这样，作业一直在作业说明书的控制下运行，直至遇到作业结束语句时，系统才停止该作业的运行。

2. 程序接口

该接口是为用户程序在执行中访问系统资源而设置的，是用户程序取得操作系统服务的唯一途径。它由一组系统调用组成，每一个系统调用都是一个能完成特定功能的子程序，每当应用程序要求操作系统提供某种服务（功能）时，便调用具有相应功能的系统调用。早期的系统调用都是用汇编语言编写的，只有在用汇编语言书写的程序中，才能直接使用系统调用；但在高级语言以及 C 语言中，往往提供了与各系统调用一一对应的库函数，这样，应用程序便可通过调用对应的库函数来使用系统调用。但在近些年所推出的操作系统中，其系统调用本身已经采用 C 语言编写，并以函数形式提供，故在用 C 语言编制的程序中，可直接使用系统调用。

1.5. UNIX 操作系统

由于本书主要以 UNIX 操作系统为例,说明操作系统的工作原理,为此,本节先对 UNIX 系统做一简要介绍。

1.5.1 UNIX 系统的产生与发展

UNIX 是一个强大的多用户、多任务操作系统,支持多种处理器架构,最早由 Ken Thompson、Dennis Ritchie 和 Douglas McIlroy 于 1969 年在 AT&T 的贝尔实验室开发。从总体来看,UNIX 的发展可以分为三个阶段:

(1) 第一阶段为 UNIX 的初始发展阶段。1969 年 Ken Thompson 在贝尔实验室创造了 UNIX 操作系统,刚开始运行在一台 PDP-7 计算机上,只在实验室内使用并完善它,这个阶段 UNIX 从版本 1 发展到了版本 6。

值得注意的是,早期的 UNIX 是用汇编语言写成的,以至于在 1970 年将 UNIX 移植到 PDP-11/20 上的时候花费了大量工作。在这个阶段里最重要的事件可以算 UNIX 的作者使用 C 语言对 UNIX 的源代码重新改写,使 UNIX 具有了可移植性。

由于 UNIX 是被作为研究项目,其他科研机构 and 大学的计算机研究人员也希望能得到这个系统,以便进行自己的研究。AT&T 以分发许可证的方法,对 UNIX 仅仅收取很少的费用,大学和研究机构就能获得 UNIX 的源代码以进行研究。UNIX 的源代码被散发到各个大学,一方面使得科研人员能够根据需要改进系统,或者将其移植到其他的硬件环境中去,另一方面培养了懂得 UNIX 使用和编程的大量的学生,这使得 UNIX 的普及更为广泛。

(2) 第二阶段为 80 年代,这是 UNIX 的丰富发展时期。在 UNIX 发展到了版本 6 之后,一方面 AT&T 继续发展内部使用的 UNIX 版本 7,同时也发展了一个对外发行的版本,但改用 System 加罗马字母作版本号来称呼它。System III 和 System V 都是相当重要的 UNIX 版本。此外,其他厂商,以及科研机构都纷纷改进 UNIX,其中以加州大学伯克利分校的 BSD 版本最为著名,从 4.2BSD 中也派生出了多种商业 UNIX 版本比如 Solaris、HP-UX、IRIX、AIX、SCO 等等。

(3) 第三阶段是 UNIX 的完善阶段,从 90 年代开始到现在。当 AT&T 推出 System V Release 4 (第五版本的第四次正式发布产品)之后,它和伯克利的 4.3BSD 已经形成了当前 UNIX 的两大流派。此时,AT&T 认识到了 UNIX 的价值,因此它起诉包括伯克利在内的很多厂商,伯克利不得不推出不包含任何 AT&T 源代码的 4.4BSD Lite,这次司法起诉也使很多 UNIX 厂商从 BSD 转向了 System V 流派。

虽然 UNIX 已经非常开放,但在 80 年代其内核代码也不是随意就可以得到的。最容易得到的代码是 Minix,用于教学目的而编写的一个系统,这远不是一个成熟的系统。于是芬兰的 Linus 决定自己编写一个独立的操作系统,在 Internet 上发布了一个通告。这个称为 Linux 的操作系统在 Internet 上众多爱好者的帮助下迅速开发出来,并取得了巨大的成功。Linux 和其他 UNIX 的源码完全无关,严格来讲只能算仿制品。但 Linux 的开发者来自整个 Internet,具有各种 UNIX 系统的背景,因此 Linux 也集中了各种 UNIX 的优点,从性能上与商业产品毫不逊色。

经过长期的发展和完善,UNIX 目前已成长成为一种主流的操作系统技术和基于这种技术的产品大家族。由于 UNIX 具有技术成熟、可靠性高、网络和数据库功能强、伸缩性突出和开放性好等特色,可满足各行各业的实际需要,特别能满足企业重要业务的需要,已经成为

主要的工作站平台和重要的企业操作平台。UNIX 曾经是服务器操作系统的首选，占据最大市场份额，但最近在跟 WINDOWS 以及 Linux 的竞争中有所失利。

1.5.2 UNIX 系统的特点

UNIX 系统在相当短的时期内取得这样大的成功，有其内部和外部原因。从外部观察，UNIX 问世之日，正是人们开始普遍使用分时操作系统，并正在寻求一个功能齐备、使用方便、大小适中的系统之时，所以 UNIX 生逢其时。另外，这一系统是在 PDP-11 系列机上开发出来的，而这种机型在当时已经得到广泛应用，并曾占据小型机的主要市场，这就为 UNIX 的广泛配置创造了物质条件。这些外部原因都非常重要，但是决定性的因素还是内部原因，即：UNIX 本身的性能和特点决定了它的成败。

下面简要分析 UNIX 系统的某些特点，以供读者在学习 UNIX 系统和本教程时进一步思考。

1. 精巧的核心与丰富的实用层

UNIX 系统在结构上分成两大层：内核层和核外程序。内核层小巧，而核外程序丰富。内核层包括进程管理、存储管理、设备管理、文件系统管理等几个部分。UNIX 的设计者对内核功能作了仔细考虑，内核层设计得非常精干简洁，其主要算法经过反复推敲，对其中包含的数据结构和程序进行了精心设计。因此，核心层只需占用很小的存储空间，并能常驻内存，以保证系统以较高的效率工作。

核外程序是那些能从内核层分离出来的部分，它们以核外程序的形式出现并在用户环境下运行。这些核外程序包含有丰富的语言处理程序。UNIX 支持十几种常用程序设计语言的编译和解释程序，如 C、APL、FORTRAN77、PASCAL、SNOBOL、COBOL、BASIC、ALGOL68 等。还包括其他操作系统常见的实用程序，如编辑程序、调试程序、有关系统状态监控和文件管理的实用程序等。UNIX 还有一组强有力的软件工具，用户能比较轻松地使用它们来开发新的软件。另外，UNIX 的命令解释程序 Shell 也属于核外程序。正是这些核外程序给用户提供了相当完备的使用环境。

UNIX 系统的设计特别注意了内核和核外程序的有机结合。内核层向核外程序提供充分而强有力的支持。核外程序则以内核为基础，最终都使用由核心层提供的底层服务，灵活而恰到好处地运用了内核的支持。核心层和核外程序两者结合起来作为一个整体，向用户提供各种良好的服务。

2. 使用灵活的用户界面

UNIX 向用户提供了命令接口和程序接口两种用户界面。界面的使用非常方便、功能齐备、清晰而灵活，易于按用户需要进行扩充和修改。

UNIX 命令接口是一种命令程序设计语言 Shell。Shell 首先是一种命令语言。UNIX 的 200 多条命令对应着 200 个实用程序。Shell 也是一种程序设计语言。它具有许多高级语言所拥有的控制流能力，如 if、for、while、until、case 语句，以及对字符串变量的赋值、替换、传递参数、命令替换等能力。对初学者而言，这种语言不难学习，使用比较方便。对熟练用户而言，则有较多高级手段可以利用，使他们能够高效率地在系统上进行工作。

UNIX 程序接口被称为系统调用，它是用户在编写程序时可以使用的界面。UNIX 不但在汇编语言，而且也在 C 语言中向用户提供这种界面。用户可以把它们看成是 C 语言的一部分加以利用。我们将在后续相关章节中介绍若干主要系统调用的功能及其内部实现方法。

3. 树形结构的文件系统

UNIX 系统采用树型目录结构来组织各种文件及文件目录。这样的组织方式有利于存储空间的分配及快速查找文件，也可以为不同用户的文件提供文件共享和存取控制的能力，且保证用户之间安全有效的合作。

4. 文件和设备统一看待

UNIX 系统中的文件是无结构的字节序列。在缺省情况下，文件都是顺序存取的，但用户如果需要的话，也可为文件建立自己需要的结构，用户也可以通过改变读 / 写指针对文件进行随机存取。

另一方面，UNIX 将普通数据文件、目录文件和外围设备都统一作为文件处理，外围设备如同磁盘上的普通文件一样被访问、共享和保护。用户不必区分文件和设备，也不需要知道设备的物理特性就能访问它。例如系统中行式打印机对应的文件名是 `/dev/lp`。用户只要用文件的写操作 (write) 就能把它的从打印机上输出。这样在用户面前，文件的概念简单了，使用也方便了。

5. 良好的移植性

UNIX 的所有核外程序和内核的 90% 代码是用 C 语言写成的，这使得 UNIX 系统易于理解、修改和扩充，成为一个可移植的操作系统。

操作系统的可移植性带来了应用程序的可移植性，因而用户的应用程序即可用于小型机，又可用于其他的微型机或大型机。从而大大提高了用户的工作效率。

虽然 UNIX 系统取得了巨大的成功，但它也不是没有缺点的，比如：UNIX 系统的版本太多，造成应用程序的可移植性不能完全实现；UNIX 系统的核心结构不是层次的，故显得十分复杂，不易修改和扩充。但是，UNIX 系统的这些缺点比起它的成就而言是十分次要的，它的成功是辉煌的。

UNIX 操作系统的另外一些特点涉及比较具体的实现技术，我们将在有关章节中进一步介绍。

1.5.3 关于 UNIX V6++

1. UNIX V6

UNIX V6 由肯·汤普逊 (Kenneth Lane Thompson) 和丹尼斯·里奇 (Dennis MacAlistair Ritchie) 开发，由贝尔实验室于 1975 年发布。因为大部分代码都是用 C 语言完成的，并且公开了源代码，UNIX V6 因此得到广泛使用，并广泛配置在各大大学的 PDP-11 机器上。它是现代各类 UNIX 操作系统的源头，它具备了现代操作系统的绝大部分特征：进程管理和调度、内存管理、文件系统和 I/O 设备管理等。它是非常优秀的系统程序，结构清晰，短小精悍，在商用系统程序领域，它的性能是空前绝后的。在此之后，UNIX 分为众多不同流派，但它们基本上都秉承了 UNIX V6 的设计思想。

在 UNIX 幼年时，其代码免费向大学开放，这引起了学术界对 UNIX 的极大关注，澳大利亚新南威尔士大学的 John Lions 甚至逐行给出了 V6 源代码的注释，这就是著名的《莱昂氏 Unix 操作系统源代码分析》，这本非正式出版的小册子一度成为许多大学操作系统课程的教材。

2. UNIX V6++

在多年的操作系统教学实践中，笔者深感通过深入阅读分析 UNIX V6 这一经典系统的源代码来学习操作系统是非常有价值的途径。通过一个可以运行的精简内核，指导同学们随

着课程的展开修改相关模块、让学生们亲身感受系统在自己手中逐步完善的快感，这是笔者多年的梦想。

UNIX V6++是这个梦想的结晶。由于 UNIX V6 只能运行在早已消失的古老机器 PDP-11 上，学生无法做 UNIX V6 内核的实验，更无法通过修改 UNIX V6 内核来加深理解。于是，笔者萌生了将 UNIX V6 移植到 PC 机上的想法。为了能让学生更容易理解代码，又用面向对象的思想重新进行了设计，并用 C++加以实现。因为尽可能地遵循了 UNIX V6 的思想和算法，因此，我们将新系统命名为 UNIX V6++。同时，为方便学生自行修改 UNIX V6++，又专门为它设计开发了一个集成环境，学生可以用这个集成环境来修改、编译、调试 V6++ 内核。

与目前所有的实验性操作系统相比，UNIX V6++将 UNIX V6 源代码移植至 i386 体系结构上，最忠实地继承了 V6 的卓越设计思想；代码尺寸与 V6 相当，采用面向对象的程序设计风格，是一个便于学生理解、教师教学的操作系统。

由于采用面向对象的设计思想及 C++语言的实现，UNIX V6++的内核由两个重要的单体系类（Singleton）：Machine 和 Kernel 组成，分别抽象了机器实例与操作系统内核实例，其中封装了进程管理、内存管理、设备管理、文件系统、中断处理、系统调用以及硬件管理等几个模块。图 1.5 示出了该系统的整体架构及各个模块在内核中的地位以及相互间关系。

设计这两个类是出于这样的考虑：操作系统中对底层硬件的操作与内核各个模块本身的数据结构、处理逻辑这两者应当加以区分。从图 1.5 中可以直观地看出 Machine 类和 Kernel 类的界定范围：Machine 类负责系统中硬件抽象层部分，系统中硬件控制芯片，汇编语言抽象类，I386 体系结构的抽象属于硬件抽象层，它们与 Machine 类相关；而 Kernel 类负责进程管理、内存管理、设备管理、文件系统等内核模块，是对内核各个模块的数据和操作进行封装，处于系统的核心位置。

本书以 UNIX V6++为教学案例。为了便于读者学习本教程，我们在安排以下各章节内容时，既考虑了操作系统一般工作原理和概念叙述的完整性，也注意到了 UNIX 操作系统各部分的相互依赖和调用关系。后续每个章节都将以先介绍内核设计原理，后详述实现方案的方式推出该章节介绍的所有概念。

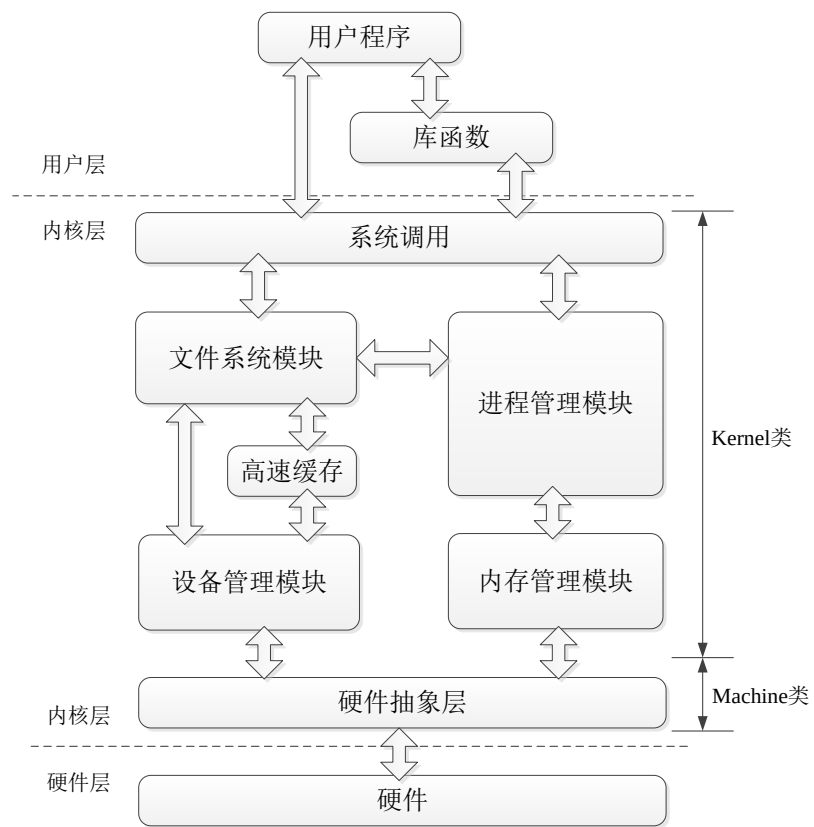


图 1.5: UNIX V6++系统内核框架